

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)**

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 — CI/CD PIPELINE DESIGN EXERCISE

**"ATHLETE-SHIELD AI:
CI/CD PIPELINE DESIGN FOR INTELLIGENT
ATHLETE INJURY RISK
PREDICTION & PERFORMANCE MONITORING"**

CI/CD Pipeline Design Exercise

Submitted in the partial fulfilment for the Course of
CSA1063 (CSA10) — SOFTWARE ENGINEERING
to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Submitted by
VENNAMPALLI BOOPATHY KHANISHK (192524311)

Under the Supervision of
Dr Anita Davamani K

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences, Chennai – 602105

AUGUST 2026

Table of Contents

Assessment Details & Code of Conduct..... 3

1. Introduction & Problem Analysis..... 4

2. Pipeline Architecture & Workflow Design 6

3. Tools & Technology Selection..... 9

4. Automated Testing Strategy 11

5. Deployment Strategy 13

6. Security, Quality Gates, Notifications & Rollback 14

7. Sample CI/CD Configuration..... 16

8. Challenges & Future Enhancements 18

9. Conclusion..... 19

References 20

Appendix A: Sample Rollback & Health-Check Scripts..... 20

Assessment Details

Field	Details
Course Code	CSA10 / CSA1063
Course Title	Software Engineering
CO Assessed	CO4
Assessment	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage	20% Total Marks: 25
Student Name	Vennampalli Boopathy Khanishk
Registration No.	192524311
Duration	60 minutes

Code of Conduct

I, Vennampalli Boopathy Khanishk (Reg. No. 192524311), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Date: _____

CO4 Statement

Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics.

1. Introduction & Problem Analysis

1.1 Assigned Problem Statement

The assigned project is the Injury Risk Prediction System for Athletes Using Performance Data, branded as Athlete-Shield AI. The system monitors athlete training and health-related performance metrics and predicts the likelihood of injury before it occurs. It consumes historical and real-time performance data, applies machine learning through a dedicated prediction engine, and returns Low/Medium/High risk scores together with actionable recommendations to coaches and athletes.

Athlete-Shield AI is composed of four independently deployable components: a React-based web/mobile portal (frontend), a Node.js/Express application server (backend), a Python-based injury-prediction inference service (ml-service), and a MySQL database. Git version control and Docker containerization for this system were established in the CO3 Assessment Tool 1 submission. This CO4 exercise extends that foundation by designing a complete CI/CD (Continuous Integration and Continuous Deployment) pipeline that automates code integration, build, test, quality analysis, packaging, and deployment for the same system.

1.2 Why This System Needs a CI/CD Pipeline

- Four heterogeneous services (React, Node.js, Python/ML, MySQL) are developed in parallel by different contributors and must be integration-tested together before every release.
- The system handles athlete health data, so every change must pass automated security and quality gates before reaching production — manual, ad-hoc deployment is not acceptable.
- Injury risk predictions directly influence coaching decisions; a regression in the ml-service must be caught by automated tests before it reaches athletes and coaches.
- Docker images already exist for each service (from the CO3 assignment); a pipeline is needed to build, scan, version, and promote those images automatically instead of manual docker-compose commands.
- Frequent hotfixes (e.g., the database-connection fix identified in the CO3 report) require a fast, reliable, and auditable path from commit to production with the ability to roll back instantly.

1.3 CI/CD Requirement Analysis by Module

Module	CI/CD-Specific Requirement
Frontend (React)	Lint, unit test, and produce an optimized static production build; containerize with Nginx; verify bundle size before deploy.
Backend (Node.js/Express)	Lint, unit + integration test against a test database, verify API contracts, build and scan Docker image.
ML Service (Python)	Run unit tests on preprocessing logic, validate the trained model's accuracy against a held-out test set before packaging, scan dependencies for CVEs.
Database (MySQL)	Validate schema migrations against a disposable test database instance; ensure init.sql / migration scripts apply cleanly.
Cross-cutting	Secrets management, image versioning/tagging, staged promotion (dev → staging → production), rollback, and stakeholder notification.

2. Pipeline Architecture & Workflow Design

2.1 Pipeline Design Philosophy

The pipeline follows a monorepo, path-filtered design: because frontend/, backend/, ml-service/, and database/ live in a single repository (as established in the CO3 folder structure), the pipeline uses path filters so that a change to only one service triggers build and test jobs for that service alone, keeping feedback fast. All four service jobs run in parallel and converge at a shared quality gate before any image is packaged. The pipeline is designed as a series of gated stages, where a stage's failure halts promotion of the change to the next stage, and every stage's result is visible on the pull request and in a central dashboard.

2.2 Pipeline Stages Overview

#	Stage	Purpose
1	Source Code Management (SCM) Trigger	A push or pull request against development/main triggers the workflow via a GitHub webhook.
2	Build & Unit Test	Each service is installed, linted, compiled/built, and unit-tested in an isolated, parallel job.
3	Code Quality & Security Analysis	Static analysis (SonarQube), dependency/image vulnerability scanning (Trivy), and audit tools run against the merged codebase.
4	Packaging	Docker images are built per service, tagged with the Git SHA and semantic version, and pushed to a container registry.
5	Deploy — Development	The updated stack is auto-deployed to a shared development environment via docker-compose for early integration feedback.
6	Deploy — Staging & Integration Testing	Images are promoted to a staging environment; integration and end-to-end tests run against the live stack.
7	Manual Approval Gate	A designated reviewer approves promotion to production after reviewing staging test results.
8	Deploy — Production	Approved images are rolled out to production using a blue-green strategy for zero-downtime cutover.
9	Post-Deployment Monitoring & Rollback	Health checks and alerting monitor the new release; a failed health check triggers automatic rollback to the last stable image.

2.3 Pipeline Diagram

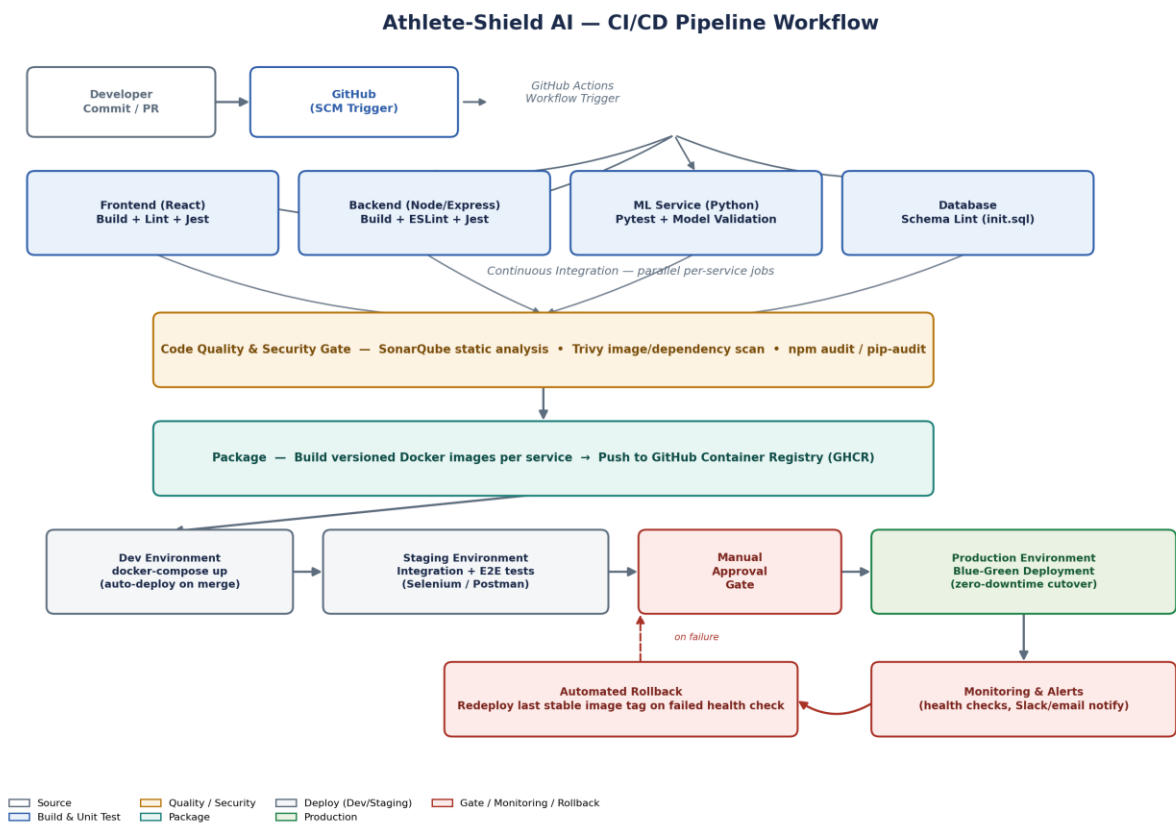


Figure 2.1: End-to-end CI/CD pipeline workflow for Athlete-Shield AI, from commit to production with automated rollback.

2.4 Workflow Narrative — From Commit to Deployment

A developer pushes a commit or opens a pull request against the development branch (continuing the Gitflow model adopted in the CO3 report). GitHub Actions detects the change and, using path filters, launches build-and-test jobs only for the services whose files changed — for example, a change confined to ml-service/ will not rebuild the frontend. Each job installs dependencies, runs linters, executes the service's unit tests, and reports results back to the pull request within minutes.

Once every triggered job succeeds, the workflow proceeds to a combined quality gate: SonarQube performs static analysis for code smells, duplication, and maintainability issues, while Trivy scans both the source dependency tree and any previously built base images for known CVEs. A merge is blocked if either tool reports a finding above the configured severity threshold (High or Critical).

After the quality gate passes, each service's Dockerfile (already defined in the CO3 submission) is built into a versioned image tagged with the short Git SHA and, on tagged releases, a semantic version (for example v1.3.0). Images are pushed to the GitHub Container Registry (GHCR). The pipeline then deploys the updated docker-compose stack to a development environment automatically, giving the team a continuously up-to-date integration environment without manual intervention.

On merges to main, or on a manually triggered release workflow, the same immutable images are promoted — never rebuilt — to staging, where integration tests (API contract checks against the backend, and Selenium-driven end-to-end tests exercising the athlete risk-prediction flow through the frontend) run against the live multi-container stack. If staging tests pass, a designated approver

reviews a summary of test results and image versions and manually approves the production promotion gate. Production deployment then uses a blue-green strategy: the new version is started alongside the currently running version, traffic is cut over once health checks pass, and the previous version is kept warm for a configurable window to allow an instant rollback.

3. Tools & Technology Selection

Tool selection favors options that integrate natively with the GitHub-hosted repository already established for Athlete-Shield AI, keep the pipeline free of vendor lock-in, and match each service's existing runtime (Node.js, Python, MySQL) from the CO3 containerization work.

Stage	Selected Tool(s)	Justification
SCM & Orchestration	GitHub + GitHub Actions	The repository is already hosted on GitHub (per the CO3 git remote setup); Actions requires no separate CI server, uses YAML workflows versioned alongside the code, and has a large marketplace of pre-built actions for Node.js, Python, and Docker.
Build — Frontend/Backend	Node.js 20 (via actions/setup-node), npm	Matches the Node.js 20-alpine base image already used in the backend Dockerfile, ensuring build-time and runtime versions stay consistent.
Build — ML Service	Python 3.11 (via actions/setup-python), pip	Matches the python:3.11-slim base image used for ml-service, avoiding version drift between CI and the container.
Unit Testing	Jest (frontend & backend), Pytest (ml-service)	Jest is the de facto standard for React/Node projects with built-in mocking and coverage; Pytest is the standard for Python ML code and integrates cleanly with scikit-learn test fixtures.
Static Code Analysis	SonarQube (Community/Cloud)	Provides a single quality dashboard across JavaScript and Python, tracks maintainability and duplication over time, and can enforce a quality gate that blocks the pipeline.
Dependency & Image Security Scanning	Trivy, npm audit, pip-audit	Trivy scans both filesystem dependencies and built container images for known CVEs in one tool, directly addressing the containerized, multi-runtime nature of this project.
Containerization	Docker, Docker Compose	Continues the containerization strategy already designed and justified in the CO3 report; no new container technology is introduced.
Container Registry	GitHub Container Registry (GHCR)	Co-located with the source repository and Actions workflows, uses the same authentication token, and avoids managing a separate registry account.
Integration / E2E Testing	Postman/Newman (API contracts), Selenium	Newman runs the same API collections developers use locally against the live backend; Selenium exercises the actual coach-dashboard risk-prediction flow described in the CO3 testing section.
Deployment Orchestration	Docker Compose (dev/staging), with a documented path to Kubernetes for production scaling	Compose matches the existing docker-compose.yml and keeps dev/staging simple; the CO3 report already identifies Kubernetes migration as a future enhancement for scaling ml-service independently.
Secrets Management	GitHub Actions Encrypted Secrets	Keeps MYSQL_ROOT_PASSWORD and other .env values (already excluded from Git via .gitignore) out of the repository while remaining available to workflow jobs.
Notifications	Slack webhook + email (GitHub Actions notification step)	Notifies the frontend, backend, and ML contributors identified in the CO3 collaboration model immediately on pipeline failure or successful production deployment.

4. Automated Testing Strategy

Testing is layered so that faster, cheaper tests run first and gate slower, more expensive tests — a failure at any layer stops the pipeline before wasting time on later stages.

4.1 Testing Layers

Layer	Scope	Runs In	Tooling
Unit Tests	Individual functions/components: React components, Express route handlers/controllers, ML preprocessing functions	Build & Unit Test stage (parallel per service)	Jest, Pytest
Model Validation Tests	Trained model's accuracy/F1-score on a fixed held-out test set must not regress below a defined threshold	ML service job, before packaging	Pytest + scikit-learn metrics
Static / Security Analysis	Code smells, duplication, known-vulnerable dependencies, container CVEs	Code Quality & Security Gate	SonarQube, Trivy, npm audit, pip-audit
API / Contract Tests	Backend REST endpoints (/api/athletes, /api/predict) match expected request/response contracts	Staging, after deployment	Postman/Newman
Integration Tests	Backend ↔ MySQL, Backend ↔ ml-service communication over the athlete-net Docker network	Staging	Newman, custom test scripts
End-to-End (E2E) Tests	Full coach-dashboard user journey: log in, submit training data, view a generated risk score	Staging	Selenium
Smoke Tests	Health-check endpoints (e.g., /health on ml-service) return healthy immediately after production cutover	Production, immediately post-deploy	curl-based health check step in the workflow

4.2 Quality Gates and Thresholds

- Unit test coverage must not fall below an agreed threshold (e.g., 80% for backend and ml-service business logic) or the build fails.
- SonarQube quality gate blocks merge on new Critical/Blocker issues or a drop in maintainability rating.
- Trivy scan fails the build on any High or Critical severity vulnerability without an accepted exception.
- The ml-service model-validation step fails the build if accuracy on the held-out set drops more than a defined tolerance versus the currently deployed model, preventing a silently worse risk-prediction model from reaching coaches and athletes.
- Staging E2E tests must pass 100% before the manual approval gate becomes available to the approver.

5. Deployment Strategy

5.1 Environment Design

Environment	Purpose	Trigger	Data
Development	Continuous integration environment for early feedback; mirrors docker-compose.yml from the CO3 submission	Automatic, on every merge to development	Seeded/synthetic athlete data
Staging	Production-like environment for integration and E2E testing before release	Automatic, on every merge to main	Anonymized copy of production-shaped data
Production	Live environment used by coaches and athletes	Manual approval after staging tests pass	Real athlete performance and injury data

5.2 Deployment Method

Development and staging use the existing Docker Compose stack (frontend, backend, ml-service, db on the athlete-net bridge network, as designed in the CO3 report), redeployed with the newly built image tags on every successful pipeline run. Production uses a blue-green deployment: a full second stack (“green”) is started alongside the currently live stack (“blue”) using the newly approved image tags, health checks are run against the green stack's endpoints (backend /api/athletes, ml-service /health), and only once all checks pass is the reverse proxy/load balancer's traffic cut over to green. The blue stack is kept running for a rollback window before being torn down.

5.3 Configuration & Secrets Across Environments

Each environment has its own set of GitHub Actions environment secrets (database credentials, ML_SERVICE_URL, registry tokens), mirroring the .env-based approach already used locally in the CO3 report but managed centrally instead of per-developer files. Environment-specific values are injected at deploy time so that the same immutable Docker images are used unmodified across dev, staging, and production — only configuration changes between environments, never the application code.

6. Security, Quality Gates, Notifications & Rollback

6.1 Security Practices in the Pipeline

- All secrets (database passwords, registry tokens, Slack webhook URL) are stored as GitHub Actions encrypted secrets, never committed to the repository, continuing the .gitignore/.env.example discipline from the CO3 report.
- Trivy scans both source dependencies and built container images before any image is pushed to the registry or deployed.
- Docker images are built from the minimal Alpine/Slim base images already selected in the CO3 Dockerfiles, reducing the attack surface scanned at each stage.
- Branch protection rules (identified as a future improvement in the CO3 report) are enforced: main and development require passing CI checks and at least one review before merge.
- Production deployment credentials are scoped separately from development/staging credentials, so a compromised development token cannot be used to deploy to production.

6.2 Notifications

The workflow posts a Slack notification and an email to the project's frontend, backend, and ML contributors on: pipeline failure at any stage (with a link to the failing job's logs), successful staging deployment (prompting the approver to review), and successful or rolled-back production deployment. This keeps every contributor identified in the CO3 collaboration model aware of the pipeline's state without needing to check the GitHub Actions dashboard manually.

6.3 Rollback Mechanism

Because every deployment uses an immutable, uniquely tagged image, rollback is a redeploy operation rather than a rebuild. If the post-deployment smoke test or health check on the newly cut-over production stack fails, the pipeline automatically re-points the load balancer back to the still-running blue stack and redeploys the last known-good image tag, and a rollback notification is sent immediately. Because the blue stack is kept warm for a defined window after cutover rather than torn down immediately, this rollback completes in seconds rather than requiring a fresh build.

7. Sample CI/CD Configuration

The following simplified GitHub Actions workflow (.github/workflows/ci-cd.yml) illustrates how the stages above are expressed as pipeline configuration for the Athlete-Shield AI monorepo. Per-service jobs use path filters so unrelated services are not rebuilt on every change.

```
name: Athlete-Shield AI CI/CD

on:
  push:
    branches: [development, main]
  pull_request:
    branches: [development, main]

jobs:
  backend-build-test:
    if: contains(github.event.head_commit.modified, 'backend/')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: cd backend && npm ci
      - run: cd backend && npm run lint
      - run: cd backend && npm test -- --coverage

  ml-service-build-test:
    if: contains(github.event.head_commit.modified, 'ml-service/')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: cd ml-service && pip install -r requirements.txt
      - run: cd ml-service && pytest --cov
      - run: cd ml-service && python validate_model.py --min-accuracy 0.85

  quality-and-security:
    needs: [backend-build-test, ml-service-build-test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: SonarSource/sonarqube-scan-action@v2
      - uses: aquasecurity/trivy-action@0.24.0
        with: { scan-type: 'fs', severity: 'HIGH,CRITICAL', exit-code: '1' }

  package-and-push:
    needs: quality-and-security
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: ${{ github.actor }}, password: ${{ secrets.GITHUB_TOKEN }} }
      - run: docker build -t ghcr.io/org/athlete-shield-backend:${{ github.sha }} ./backend
      - run: docker push ghcr.io/org/athlete-shield-backend:${{ github.sha }}

  deploy-staging:
```

```
needs: package-and-push
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
environment: staging
steps:
  - run: ./scripts/deploy.sh staging ${github.sha}
  - run: newman run tests/api-collection.json --env-var base_url=$STAGING_URL

deploy-production:
needs: deploy-staging
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
environment:
  name: production
  url: https://athlete-shield.example.com
steps:
  - run: ./scripts/deploy-blue-green.sh production ${github.sha}
  - run: ./scripts/health-check.sh || ./scripts/rollback.sh
```

Figure 7.1: Simplified representative GitHub Actions workflow implementing the designed pipeline stages.

8. Challenges & Future Enhancements

8.1 Anticipated Challenges

- Cross-service integration failures: since frontend, backend, ml-service, and database are tested independently before the staging stage, a contract mismatch (e.g., a changed `/api/predict` response shape) may only surface during staging integration tests, requiring clear contract documentation between services.
- Build time growth: as identified in the CO3 report, the ml-service's scikit-learn/pandas/numpy dependencies already produce a large image; without Docker layer caching in CI, every pipeline run risks slow rebuilds.
- Flaky end-to-end tests: Selenium-based dashboard tests are sensitive to timing and environment differences between staging and a developer's laptop, which can produce false-positive pipeline failures.
- Secrets sprawl across environments: maintaining separate, correctly-scoped secrets for development, staging, and production (extending the `.env` discipline from CO3) requires ongoing governance as the team grows.
- Model rollback complexity: rolling back the ml-service container also rolls back its bundled model artifact, so a poorly performing model requires the same immutable-image rollback path as a code defect.

8.2 Future Enhancements

- Introduce a Docker layer cache (e.g., GitHub Actions cache backend for Buildx) to speed up repeated ml-service builds, directly addressing the image-size/build-time challenge noted in the CO3 report.
- Adopt multi-stage Docker builds for ml-service to discard build-time dependencies from the final runtime image, as already suggested as a CO3 future enhancement.
- Migrate staging and production orchestration from Docker Compose to Kubernetes, enabling independent auto-scaling of the ml-service under heavy load.
- Add canary deployment as an alternative to blue-green for production, gradually shifting a small percentage of coach/athlete traffic to the new version before a full cutover.
- Automate database schema migrations (replacing the single `init.sql`) so schema changes are version-controlled and applied incrementally as part of the pipeline, rather than manually.
- Add a scheduled nightly pipeline run against the full test suite and the latest dependency versions to catch newly disclosed CVEs even when no code has changed.

9. Conclusion

This exercise has designed a complete CI/CD pipeline for the Athlete-Shield AI Injury Risk Prediction System, extending the Git branching strategy and Docker containerization already established in the CO3 Assessment Tool 1 submission into a fully automated path from commit to production. The pipeline covers source-triggered parallel builds and unit tests for each of the system's four components, a combined static-analysis and security-scanning quality gate, versioned Docker image packaging to a container registry, staged promotion through development and staging environments with integration and end-to-end testing, a manual approval gate, and a blue-green production deployment with automated health-check-triggered rollback.

Tool selections (GitHub Actions, SonarQube, Trivy, GHCR, Selenium/Newman) were chosen to integrate directly with the existing GitHub-hosted repository and the Node.js, Python, and MySQL runtimes already containerized for this project, avoiding the introduction of unrelated infrastructure. Together, these design decisions directly satisfy the CO4 outcome of implementing robust testing, quality assurance, and DevOps practices with an emphasis on security, reliability, and ethics — while building coherently on the version-control and containerization foundation documented in the CO3 report.

References

- [1] GitHub, "GitHub Actions Documentation," GitHub Inc. [Online]. Available: <https://docs.github.com/actions>
- [2] SonarSource, "SonarQube Documentation," SonarSource. [Online]. Available: <https://docs.sonarsource.com>
- [3] Aqua Security, "Trivy Documentation," Aqua Security. [Online]. Available: <https://aquasecurity.github.io/trivy>
- [4] Docker Inc., "Docker Compose File Reference," Docker. [Online]. Available: <https://docs.docker.com/compose/compose-file/>
- [5] M. Fowler, "BlueGreenDeployment," martinfowler.com, 2010. [Online]. Available: <https://martinfowler.com/bliki/BlueGreenDeployment.html>
- [6] V. Driessen, "A Successful Git Branching Model," nvie.com, 2010. [Online]. Available: <https://nvie.com/posts/a-successful-git-branching-model/>
- [7] Postman, "Newman CLI Collection Runner," Postman Inc. [Online]. Available: <https://learning.postman.com/docs/collections/using-newman-cli/>

Appendix A: Sample Rollback & Health-Check Scripts

The scripts below illustrate the health-check and automated-rollback logic referenced in Section 6.3, called by the deploy-production job of the workflow shown in Figure 7.1.

A.1 health-check.sh

```
#!/bin/bash
# Polls the newly deployed 'green' stack's health endpoints
set -e
MAX_RETRIES=10
for i in $(seq 1 $MAX_RETRIES); do
  BACKEND_OK=$(curl -sf http://green.internal:5000/api/athletes -o /dev/null && echo 1 || echo 0)
  ML_OK=$(curl -sf http://green.internal:8000/health | grep -q "status": "ok" && echo 1 || echo 0)
  if [ "$BACKEND_OK" = "1" ] && [ "$ML_OK" = "1" ]; then
    echo 'Green stack healthy — proceeding with traffic cutover'
    exit 0
  fi
  echo "Attempt $i/$MAX_RETRIES failed, retrying in 6s..."
  sleep 6
done
echo 'Green stack failed health checks'
exit 1
```

A.2 rollback.sh

```
#!/bin/bash
# Invoked automatically when health-check.sh exits non-zero
set -e
echo 'Rolling back: re-pointing load balancer to blue (last stable) stack'
./scripts/switch-traffic.sh blue
echo 'Notifying team of rollback'
curl -X POST -H 'Content-Type: application/json' \
  -d '{"text": "Production rollback triggered for Athlete-Shield AI — green stack failed health checks."}' \
  $SLACK_WEBHOOK_URL
exit 0
```

A.3 Appendix Summary

Because health-check.sh and rollback.sh operate purely on already-built, already-tagged images and a simple traffic-switch operation, the entire rollback path completes without any rebuild step, keeping mean-time-to-recovery low for a system where injury-risk predictions must remain continuously available to coaches and athletes.